

RTL Obfuscation

本项目用于加密 SystemVerilog RTL 中的名称，例如信号、实例、参数、类型、module 和 interface 名称。加密只改变名称，不改变 RTL 的预期功能，并且可以恢复原始源码。

请从仓库根目录运行。需要 Python 3.10 或更高版本，并确保当前 Python 环境已经安装 PySlang 11.x。无需安装本项目；如果目标服务器无法联网安装 PySlang，请先阅读 [PySlang 源码编译与离线部署指南](#)。可先查看命令帮助：

```
python rtl_encrypt.py --help
python rtl_decrypt.py --help
```

文档导航

- [PySlang 源码编译与离线部署指南](#)：服务器无法联网时准备运行环境。
- [SystemVerilog 可加密类型表](#)：查看 `--category` 可以选择的内容。
- [Formal 验证流程](#)：加密完成后，按需独立检查原始 RTL 与加密 RTL 的功能等价性。
- [开发文档索引](#)：只面向项目维护者，不是用户操作必读内容。

加密模式

模式	输入	加密范围	加密内容
单文件	一个 .sv 文件及源码根目录	该输入文件	不改变 module 端口，只加密内部名称
filelist	filelist 及源码根目录	filelist 中的全部文件	不提供 <code>--top</code> 时不改变 module 端口；提供 <code>--top</code> 后会同时加密子 module 端口和跨 module 引用
project-root	源码根目录及 top module	top module 及其使用的全部源码	只保留顶层 module 名称和端口，加密子 module 端口以及跨 module 使用的接口、参数和类型

不提供 `--category` 时，单文件和不带 `--top` 的 filelist 默认加密 13 类常用内部名称，包括信号、实例、参数、结构体、函数和任务等。它们不会改变其他 module 调用当前 module 时使用的名称。

filelist 可以增加 `--top`。filelist 中的全部文件仍会处理；同时，工具会在该 top 使用到的范围内一致地修改子 module 名称、子 module 端口、interface、参数和类型等跨 module 名称。

project-root 必须提供 `--top`。工具会自动找到该 top 使用的源码，并默认处理当前支持的全部 19 类名称。加密后，外部仍可通过原来的 top module 名称和端口连接设计；不应再单独调用已经改名的子 module、interface 或类型。

当前版本会保留 top module 内部直接声明的 interface 实例名；interface 类型和成员仍会加密。

使用 `--category` 可以只选择指定的加密内容。所有可选名称见 [SystemVerilog 可加密类型表](#)。

输出文件和加密率

`--output-dir` 是必填参数，用于存放加密后的 RTL。目标目录必须尚未存在，工具会在成功后一次性生成它。

如果不指定报告路径，工具会把两个报告放在加密目录内：

```
<output-dir>/mapping.json
<output-dir>/metrics.json
<output-dir>/mapping_table.csv
<output-dir>/encryption_summary.txt
```

- `mapping.json`：用于恢复原始名称，同时记录本次加密结果。
- `metrics.json`：记录实际加密覆盖率。
- `mapping_table.csv`：用表格列出每个实际替换的文件名、模块名、作用域、加密类型、原名和替换后名。
- `encryption_summary.txt`：用文字列出实际加密率、加密行数、总代码行数和实际加密类型。

CSV 中的作用域会使用简短形式，例如 `type:pair_t`、`task:select_value` 和 `generate:line 42`；不会写入完整的 `generate` 语句。

只有希望把报告放到其他位置时，才需要使用 `--map` 或 `--metrics`。可以只指定其中一个，另一个仍会写入默认位置。`mapping_table.csv` 和 `encryption_summary.txt` 始终写入 `<output-dir>`。

默认会处理当前范围内全部可加密名称。使用下面的参数可以控制加密率：

```
--encryption-rate 0.35
```

取值必须满足 $0 < \text{RATE} \leq 1$ ，其中 1 表示全部加密。小于 1 时，工具根据可加密名称影响到的有效代码行选择接近目标比例的名称，因此它不是标识符数量的精确比例。

单文件加密

基础命令

```
python rtl_encrypt.py \
--input <输入文件> \
--source-root <源码根目录> \
--output-dir <加密输出目录>
```

- `--input`：要加密的一个 `.sv` 文件，可写相对于 `--source-root` 的路径。
- `--source-root`：输入文件和相对 `include` 使用的根目录。
- `--output-dir`：加密结果目录，运行前不能存在。

示例与架构

示例文件为 `rtl_samples/11_supported_obfuscation.sv`，一个文件内包含两个 module：

```
sample11_supported_obfuscation
└── sample11_helper
```

它包含参数、信号、类型、function、task、generate 和 module 实例，适合查看默认 13 类内部名称的加密效果。

```
single_work="$(mktemp -d /tmp/rtl-obfuscation-single.XXXXXXX)"
python rtl_encrypt.py \
--input 11_supported_obfuscation.sv \
--source-root rtl_samples \
--output-dir "$single_work/gate"
```

加密报告位于 `$single_work/gate/mapping.json` 和 `$single_work/gate/metrics.json`。

Filelist 多文件加密

基础命令

```
python rtl_encrypt.py \
--filelist <文件清单.f> \
--source-root <源码根目录> \
--output-dir <加密输出目录>
```

- `--filelist`: 列出输入 `.sv/.svh` 文件的 `.f` 文件，可写相对于 `--source-root` 的路径。
- `--source-root`: filelist、源码和相对 include 使用的根目录。
- `--output-dir`: 加密结果目录，运行前不能存在。
- `--top`: 可选。不提供时只处理默认 13 类内部名称；提供后还会一致地修改该 top 使用到的子 module 端口和跨 module 名称。

示例与架构

示例使用 `rtl_samples/example_fifo/design.f`。filelist 按编译顺序列出四个文件：

```
fifo_top
├── fifo_if
├── fifo_ctrl
├── fifo_if
└── fifo_storage
```

下面提供 `--top fifo_top`，因此会处理 filelist 中的全部文件，并自动加密子 module 端口、interface、参数和其他跨 module 名称；`fifo_top` 的名称和对外端口保持不变。

```
filelist_work="$(mktemp -d /tmp/rtl-obfuscation-filelist.XXXXXXX)"
python rtl_encrypt.py \
--filelist design.f \
--source-root rtl_samples/example_fifo \
--top fifo_top \
--output-dir "$filelist_work/gate"
```

Project-root 项目加密

基础命令

```
python rtl_encrypt.py \  
--source-root <项目根目录> \  
--top <顶层模块名> \  
--output-dir <加密输出目录>
```

- `--source-root`: RTL 项目根目录。
- `--top`: 项目对外使用的顶层 module 名称。
- `--output-dir`: 加密结果目录, 运行前不能存在。

不提供 `--input` 或 `--filelist`、同时提供 `--source-root` 和 `--top` 时, 工具会自动进入项目加密模式, 从 `top` 开始找到实际使用的源码。

示例与架构

示例继续使用 `rtl_samples/example_fifo`, 架构与上一个示例相同。该命令默认处理当前支持的全部 19 类名称, 只保留 `fifo_top` 的名称和对外端口:

```
project_work="$(mktemp -d /tmp/rtl-obfuscation-project.XXXXXX)"  
python rtl_encrypt.py \  
--source-root rtl_samples/example_fifo \  
--top fifo_top \  
--output-dir "$project_work/gate"
```

如需把加密率控制在接近 35%, 只需在命令末尾增加:

```
--encryption-rate 0.35
```

解密

加密时未指定 `--map`, 映射报告默认位于 `<output-dir>/mapping.json`。恢复命令为:

```
python rtl_decrypt.py \  
--map <加密输出目录>/mapping.json \  
--gate-dir <加密输出目录> \  
--output-dir <恢复输出目录>
```

- `--map`: 加密时生成的 `mapping.json`。
- `--gate-dir`: 加密 RTL 所在目录。
- `--output-dir`: 恢复后的源码目录, 运行前不能存在。

恢复上面的 project-root 示例:

```
python rtl_decrypt.py \
--map "$project_work/gate/mapping.json" \
--gate-dir "$project_work/gate" \
--output-dir "$project_work/restored"
```

恢复成功后，四个 SystemVerilog 源文件与加密前逐字节一致。

如果还需要保存一份恢复结果报告，可以增加 `--report <□□□□.json>`。RTL 功能等价验证是独立步骤，参见 [Formal 验证流程](#)。

常用可选参数

选项	用法
<code>--include-dir PATH</code>	添加 include 目录，可重复使用；相对路径以源码根目录为基准。
<code>--define NAME[=VALUE]</code>	添加预处理宏，可重复使用，例如 <code>--define SYNTHESIS</code> 。
<code>--category NAME</code>	只加密指定类型，可重复使用；不提供时使用当前模式的默认范围。
<code>--encryption-rate RATE</code>	控制加密率，必须大于 0 且不大于 1。
<code>--name-length N</code>	设置新名称长度，最小为 4，默认值为 20。
<code>--map PATH</code>	把映射报告写到指定文件；省略时写入 <code><output-dir>/mapping.json</code> 。
<code>--metrics PATH</code>	把覆盖率报告写到指定文件；省略时写入 <code><output-dir>/metrics.json</code> 。
<code>--report PATH</code>	解密时把恢复结果报告写到指定文件；省略时只输出恢复后的 RTL。

可加密内容和 `--category` 示例见 [SystemVerilog 可加密类型表](#)。开发与维护信息见 [开发文档索引](#)。